Amazon Bedrock Model Multiplexer — Specification

Weighted Routing of Converse API Requests to Model Pools
with Post-Inference Refusal Detection

Implementation-Agnostic Behavioral Specification

Overview

The **Amazon Bedrock Model Multiplexer** routes Amazon Bedrock Converse API requests across a pool of models using weighted random selection, per-model circuit breakers, cross-model retry with failover, post-inference refusal detection, and real-time health monitoring. The system comprises six logical subsystems:

1. **Multiplexer** — orchestrator: owns model pools, delegates to the Request Handler, publishes events
2. **Request Handler** — per-request state machine: retry loop with cross-model failover and refusal-based re-routing
3. **Model Wrapper** — model adapter: stamps `modelId`, enforces timeout, records circuit breaker outcomes
4. **Refusal Classifier** — opt-in ONNX binary classifier: detects refusals in model responses, triggers cross-model retry
5. **Circuit Breaker / Circuit Breaker Manager** — per-model three-state breaker (Closed → Open → Half-Open)
6. **Utilities** — weighted selection, error classification, health checks, validation, tracing, timers

Invariants:

- Every model pool has ≥ 1 primary (non-fallback) model.
- Model IDs are unique across all pools.
- A single Circuit Breaker Manager is the source of truth for all breaker state.
- The multiplexer does not mutate the caller's input beyond stamping `modelId`.

Data Model

The multiplexer operates on the Amazon Bedrock Converse API request and response structures. The caller provides a complete Converse request *except* `modelId`, which the multiplexer stamps based on routing decisions.

Core Data Types

Type	Kind	Purpose
`MultiplexerInput`	Record	Converse API request with all fields except `modelId` (stamped by multiplexer)
`MultiplexerConfig`	Record	Top-level configuration: model list, timeout, max retries, tracing, circuit breaker settings
`ModelConfiguration`	Record	Per-model config: `modelId` (string), `weight` (number), `isFallback` (boolean)
`MultiplexerStats`	Record	Aggregate stats: success/rateLimit/failFast/refusal counts + latency metrics
`ModelStats`	Record	Per-model stats: counts + averageLatency
`LatencyMetrics`	Record	Percentiles: average, p50, p95, p99, min, max
`ModelOutcome`	Record	Result of one invocation: modelId, outcomeType, latency, timestamp
`ErrorResponse`	Record	Standardized error: code, message, details
`EnhancedErrorResponse`	Record	Extends ErrorResponse with: errorType, retryable flag, retryAfterMs, recoverySuggestions
`SelectModelRequest`	Record	Input to model selection: set of skipped model IDs
`SelectModelResponse`	Record	Output: selected modelId (nullable) + isFallback flag
`ModelHealthStatus`	Record	Per-model health: circuitState, errorRate, requestsPerMinute
`SystemHealthStatus`	Record	System-wide: healthy/degraded/unhealthy counts + aggregate metrics
`ValidationResult`	Record	isValid flag + list of validation errors
`CircuitBreakerConfig`	Record	failureThreshold, recoveryTimeMs, successThreshold, failureWindowMs
`CircuitBreakerStatus`	Record	state, failureCount, successCount, lastOpenedAt, nextRetryAt
`RefusalDetectionConfig`	Record	Opt-in config: `enabled`, `modelPath`, `confidenceThreshold`, `retryOnRefusal`
`ClassificationResult`	Record	`isRefusal` (boolean), `confidence` (float 0–1), `latencyMs`

Enumerations

Enum	Values
`OutcomeType`	`SUCCESS`, `RATE_LIMIT`, `FAIL_FAST`, `REFUSAL`
`CircuitBreakerState`	`CLOSED`, `OPEN`, `HALF_OPEN`
`ErrorType`	VALIDATION, THROTTLING, MODELUNAVAILABLE, TIMEOUT, CANCELLED, CIRCUITOPEN, INTERNAL

Multiplexer (Orchestrator)

The Multiplexer is the top-level entry point. It owns two model pools (primary and fallback), a Circuit Breaker Manager, a Health Check Manager, an optional Tracer, and an optional Refusal Classifier. It publishes typed events via an observer/event-emitter pattern (see Event System section).

Operation	Inputs / Outputs	Semantics
`construct`	config → Multiplexer	Validate config, initialize subsystems, register models
`addModel`	ModelConfiguration → void	Create Model Wrapper with shared breaker, add to appropriate pool
`removeModel`	modelId → void	Destroy model, remove from pool/health/breaker
`processRequest`	MultiplexerInput → ConverseResponse	Create Request Handler, delegate, trace, emit events
`selectModel`	SelectModelRequest → SelectModelResponse	Primary pool first, then fallback; weighted random; skip open breakers
`getModel`	modelId → ModelWrapper or null	Lookup in primary then fallback pool
`reportOutcome`	ModelOutcome → void	Update stats, record latency, update health, emit events
`getStats`	→ MultiplexerStats	Aggregate across all models + latency percentiles
`resetStats`	→ void	Zero all counters, clear latency history
`getHealthStatus`	→ SystemHealthStatus	Delegate to Health Check Manager
`isHealthy`	→ boolean	True if healthyCount > 0 and unhealthyCount $< \max(1, \lfloor n/2 \rfloor)$
`classifyRefusal`	responseText → ClassificationResult or null	Delegate to Refusal Classifier; null if disabled or not loaded
`destroy`	→ void	Destroy all models, classifier, clear breakers, remove listeners

Delegate interface — The Request Handler communicates with the Multiplexer through a narrow interface of six operations: `selectModel`, `getModel`, `reportOutcome`, `emitEvent`, `classifyRefusal`, and the `refusalRetryEnabled` flag. This decouples the retry logic from the orchestrator.

Model selection algorithm (`selectFromPool`):

1. Filter out models in the `skippedModels` set (already tried for this request).
2. Filter out models where the circuit breaker disallows execution; publish `model-circuit-open-skipped` event.
3. Build weighted item list from remaining models.
4. Call `weightedRandomSelect` — returns null if list is empty.
5. Try the primary pool first; if null, try the fallback pool.

Latency percentiles — maintained via a ring buffer of the last 1 000 latencies, sorted on read:

$$p_k = \text{sorted}[\lfloor n \cdot k \rfloor], \quad k \in \{0.50, 0.95, 0.99\}$$

Request Handler — per-request state machine

Each call to `processRequest` creates a fresh Request Handler instance. It communicates with the Multiplexer via the delegate interface described above.

Retry loop (`process`):

1. Set retryCount $\leftarrow 0$, skippedModels $\leftarrow \emptyset$.
2. **while** retryCount $\leq$ maxRetries:
   a. Call `selectModel`(`skippedModels`). If null → raise error with code `NO_MODELS_AVAILABLE`.
   b. Call `getModel`(modelId). If null → add to skipped, increment retry, continue.
   c. Call `model.invoke`(input). On success → proceed to refusal check (step d).
   d. **Post-inference refusal check** (if `refusalRetryEnabled`):
      i. Extract assistant text from response via `extractResponseText`.
      ii. Call `classifyRefusal`(text). Emit `refusal-classification` event.
      iii. If classified as refusal → emit `refusal-detected` event, report Refusal outcome, add model to skipped, increment retry, continue.
      iv. If not refusal (or classifier disabled) → report Success outcome, return response.
   e. On RateLimit error → report outcome, add model to skipped, increment retry, continue.
   f. On CircuitOpen error → report outcome, add model to skipped, increment retry, continue.

g. On FailFast error → raise immediately (no retry).
h. On unknown error → re-raise the original error as-is (no wrapping).

3. If loop exhausts → raise error with code `RETRIES_EXHAUSTED`.

Refusal = recoverable outcome: a detected refusal is treated identically to a RateLimit in the retry loop — the model is skipped and a different model is selected immediately. The response is discarded. This allows the same prompt to be retried against a model that may provide a substantive answer. If all models refuse (or retries are exhausted), the last refusal response is *not* returned — instead, a `RETRIES_EXHAUSTED` error is raised.

Model Wrapper — Amazon Bedrock model adapter

Wraps an Amazon Bedrock runtime client for a single model. Owns: client instance, model configuration, circuit breaker reference, timeout setting.

Operation	Inputs / Outputs	Semantics
`construct`	config, client?, timeoutMs?, breaker?, clientConfig?	Create Bedrock runtime client with `clientConfig` passthrough
`invoke`	input, abortSignal? → {response, outcome}	Stamp modelId, check breaker, execute with timeout
`invokeStream`	input, abortSignal? → streaming response	Streaming variant via Converse Stream API; returns raw SDK response
`destroy`	→ void	Reset circuit breaker

Invoke flow:

1. `circuitBreaker.canExecute()` — if false, raise CircuitOpenError with outcome.
2. Construct a Converse API call with {...input, modelId}.
3. `executeWithTimeout`(operation, timeoutMs, abortSignal):
   - Race: operation vs. timeout timer vs. abort signal.
   - First to resolve or reject wins; others are cleaned up.
4. On success → `breaker.recordSuccess()`, return Success outcome.
5. On abort/cancellation → return FailFast outcome (no breaker recording).
6. On timeout → `breaker.recordFailure()`, return FailFast outcome.
7. On throttling error → `breaker.recordFailure()`, raise RateLimitError with outcome.
8. On other error → `breaker.recordFailure()`, raise FailFastError with outcome.

Streaming — `invokeStream` returns the raw streaming response from the Amazon Bedrock Converse Stream API. No custom iteration, chunk transformation, or token usage extraction is performed by the multiplexer; the caller consumes the stream directly using their platform's streaming primitives.

Circuit Breaker — resilience pattern

Each model gets a Circuit Breaker instance managed by a single Circuit Breaker Manager. The breaker implements a three-state finite state machine.

Default configuration:

Parameter	Default	Meaning
`failureThreshold`	5	Recent failures within sliding window to trip OPEN
`recoveryTimeMs`	30 000	Time in OPEN before transitioning to HALF_OPEN
`successThreshold`	2	Successes in HALF_OPEN to close the circuit
`failureWindowMs`	60 000	Sliding window for counting recent failures

State transitions:

From	Event	To	Action
Closed	recentFailures ≥ threshold	Open	Record `lastOpenedAt`, reset successCount
Closed	success	Closed	Clear failure history
Open	t − lastOpenedAt ≥ recoveryTimeMs	HalfOpen	Reset successCount (checked lazily on state query)
HalfOpen	success (count ≥ successThreshold)	Closed	Clear failures, reset counts
HalfOpen	any failure	Open	Record new `lastOpenedAt`

Circuit Breaker Manager — collection-level operations:

- `getBreaker`(modelId) — get-or-create with default config.
- `canExecute`(modelId) — returns true if breaker allows requests (or breaker doesn't exist yet).
- `getAllStatus`() — snapshot of all breaker states.
- `removeBreaker`(modelId), `clear`(), `resetAll`() — lifecycle management.

Failure window cleanup: on every `recordFailure` and `getRecentFailureCount`, failures older than `failureWindowMs` are pruned from the list.

Refusal Classifier — post-inference response classification

The Refusal Classifier is an **opt-in** subsystem that detects when a model returns a refusal (e.g., "I'm sorry, but I can't help with that") instead of a substantive answer. When enabled, every successful Converse API response is classified *before* being returned to the caller. If classified as a refusal, the response is discarded and the request is re-routed to a different model — the same cross-model failover used for throttling errors.

ONNX model contract:

The classifier wraps a binary ONNX model that embeds a TF-IDF vectorizer in its computation graph, so it accepts raw text strings directly — no separate tokenizer is needed.

Direction	Name	Type	Shape / Semantics
Input	`input`	tensor(string)	$[N, 1]$ — raw text to classify
Output	`label`	tensor(string)	Predicted class: "complied" or "rejected"
Output	`probabilities`	tensor(float)	$[p_{\text{complied}}, p_{\text{rejected}}]$

A response is classified as a refusal iff $p_{\text{rejected}} \geq$ `confidenceThreshold`.

Operations:

Operation	Inputs / Outputs	Semantics
`construct`	RefusalClassifierConfig → instance	Store config; session not yet loaded
`initialize`	→ void	Create ONNX inference session (async); must be called before `classify`
`classify`	responseText → ClassificationResult	Build string tensor $[1, 1]$, run inference, return {isRefusal, confidence, latencyMs}
`destroy`	→ void	Release ONNX session resources
`isReady`	→ boolean	True iff session is loaded

Response text extraction (`extractResponseText`): extracts the assistant's text from a Converse API response by concatenating all text blocks from the output message. Returns empty string if no text blocks are present.

Configuration (`RefusalDetectionConfig`):

Field	Type	Default	Semantics
`enabled`	boolean	—	Master switch; when false, no classifier is loaded
`modelPath`	string	—	Path to the `.onnx` model file
`confidenceThreshold`	number	0.5	p_{rejected} threshold for refusal classification
`retryOnRefusal`	boolean	true	Whether to retry with a different model on refusal

Lifecycle:

1. At construction, if `refusalDetection.enabled` is true, the Multiplexer creates a `RefusalClassifier` and starts async initialization of the ONNX session.
2. The `classifyRefusal` delegate method awaits initialization before the first classification call.
3. If ONNX session initialization fails, the classifier is silently disabled (graceful degradation). A warning is logged and all future `classifyRefusal` calls return null.
4. On `destroy`, the ONNX session is released.

Scope: refusal classification applies only to non-streaming (`invoke`) responses. Streaming responses (`invokeStream`) are not classified because the full response text is not available until the stream is consumed by the caller.

Error Classification — error taxonomy

All AWS SDK errors are classified into two layers: a coarse `OutcomeType` for the retry loop, and a fine `ErrorType` for diagnostics.

Operation	Inputs / Outputs	Semantics
`classifyError`	error → OutcomeType	Throttling errors → RateLimit; thing else → FailFast
`classifyErrorType`	error → ErrorType	Fine-grained 10-variant classificati error name/message
`isThrottlingError`	errorName → boolean	Matches 6 throttling error names (s low)
`toErrorResponse`	error → ErrorResponse	Normalize to {code, message, details
`createEnhancedError`	error, errorType?, modelId? → EnhancedErrorResponse	Full diagnostic error with recovery s tions
`getRecoverySuggestions`	ErrorType → list of strings	Human-readable recovery advice per type
`getErrorMessage`	error → string	User-friendly message by AWS error

Throttling error names (any of these → RateLimit):

`ThrottlingException`	`TooManyRequestsException`	`ServiceQuotaExceededException`
`LimitExceededException`	`RequestLimitExceeded`	`RateLimitExceeded`

Failover semantics: the multiplexer does *not* perform timed retries or backoff. On a recoverable outcome (RateLimit, CircuitOpen, or Refusal), the failing model is skipped and a different model is selected immediately with zero delay. This is cross-model failover, not same-model retry.

Health Monitoring

The Health Check Manager tracks per-model metrics (request timestamps, response times, success/failure counts) in a sliding window (default 60 s, max 1 000 entries per model).

Model health: a model is *healthy* iff circuit state $\neq$ Open AND errorRate < 0.5.

System health:

- *Healthy model*: isHealthy AND circuit = Closed.
- *Degraded model*: isHealthy AND circuit $\neq$ Closed (e.g. HalfOpen).
- *Unhealthy model*: not isHealthy.
- System is healthy iff unhealthyCount $< \max(1, \lfloor \text{totalModels}/2 \rfloor)$ AND healthyCount > 0.
- The $\max(1, \ldots)$ ensures a single healthy model reports healthy: without it, $\lfloor 1/2 \rfloor = 0$ makes the condition $0 < 0$ (always false).

Health endpoint — a formatter for external consumers should provide:

- `getSimpleHealth`() → {`status`: "healthy" or "unhealthy", `code`: 200 or 503} — suitable for load balancers.
- `getDetailedHealth`() → full system snapshot with per-model detail and aggregate metrics (totalRequests, successRate, averageLatencyMs, p99LatencyMs).

Weighted Selection — model routing

Operation	Inputs / Outputs	Semantics
`weightedRandomSelect`	list of (item, weight) → item or null	Draw one item proportional to weight
`createWeightedItem`	item, weight → (item, weight)	Constructor helper

Algorithm: given items with weights $w_1, \ldots, w_n$, total $W = \sum w_i$:

$$r \sim \mathcal{U}(0, W); \quad \text{select item } k \text{ where } \sum_{i=1}^{k-1} w_i < r \leq \sum_{i=1}^{k} w_i$$

Returns null if the list is empty or $W \leq 0$.

Observability

Tracing

The Tracer provides optional distributed tracing (e.g. AWS X-Ray). If the tracing backend is unavailable, all operations silently no-op.

Operation	Inputs / Outputs	Semantics
`isEnabled`	→ boolean	True iff tracing is configured and the backend is loaded
`putAnnotation`	key, value → void	Add indexed annotation to current trace segment
`putMetadata`	key, value → void	Add metadata (only if `captureBodies` is enabled)
`traceOperation`	name, operation, metadata? → result	Create subsegment, execute, annotate outcome/latency, close
`generateRequestId`	→ string	Unique request identifier (e.g. `req_{timestamp}_{random}`)

Tracing configuration: `enabled` (boolean), `serviceName` (default "bedrock-multiplexer"), `captureBodies` (boolean), `captureModelSelection` (boolean). When `captureModelSelection` is true, model selection gets its own subsegment with skipped/available model counts.

Event System

The Multiplexer publishes typed events via an observer pattern. Consumers subscribe to events for monitoring, logging, or integration with external systems.

Event	Payload
`request`	(input)
`success`	(null, outcome: ModelOutcome)
`error`	(error: ErrorResponse or null, outcome: ModelOutcome)
`model-circuit-open-skipped`	(modelId)
`model-added` / `model-removed`	(modelId)
`stats`	(stats: MultiplexerStats)
`stats-reset`	()
`model-selected`	(modelId, isFallback, skippedCount)
`model-invocation-start`	(modelId, requestId)
`model-invocation-complete`	(modelId, requestId, latencyMs)
`refusal-detected`	(modelId, confidence, responseText)
`refusal-classification`	(modelId, isRefusal, confidence, latencyMs)

Implementation note: ensure that unhandled `error` events do not cause the process to crash. Register a default no-op handler if your platform's event system raises on unhandled error events.

Input Validation

Three validation scopes, each producing a `ValidationResult` (isValid flag + list of errors):

Scope	Key Rules
Model config	modelId required (non-empty string), weight $\in [0, 10000]$, isFallback required
Multiplexer config	models ≥ 1, unique IDs, ≥ 1 primary, timeout $\in [1000, 300000]$ ms, retries $\in [0, 10]$
Circuit breaker config	failureThreshold $\in [1, 100]$, recoveryTimeMs $\in [1000, 300000]$, successThreshold $\in [1, 20]$, failureWin

Helper operations: `formatValidationErrors`(result) → human-readable string; `assertValid`(result) raises on failure.

Timer — timeout primitive

A cancellable timeout abstraction with operations: `start`(ms), `cancel`(), `isRunning`(). Calling `start` while running cancels the previous timer first. Implementations should use platform-appropriate timer primitives.

Request Lifecycle

End-to-end pseudocode for a single `processRequest` call:

```
// 1. Caller invokes multiplexer
response = multiplexer.processRequest(input)

// Internal flow:
// 2. Multiplexer creates a new RequestHandler(input, delegate, maxRetries)
// 3. RequestHandler.process() enters retry loop:
//    a. delegate.selectModel({skippedModels})
//       → selectFromPool(primaryModels, skipped)   // weighted random, skip open breakers
//       → if null: selectFromPool(fallbackModels, skipped)
//       → if null: raise NO_MODELS_AVAILABLE
//    b. delegate.getModel(modelId)
//    c. model.invoke(input)
//       → check circuitBreaker.canExecute()
//       → Converse API call with {...input, modelId}
//       → race: operation vs timeout vs abort signal
//       → on success: breaker.recordSuccess(), return {response, outcome}
//       → on error:   breaker.recordFailure(), raise typed error with outcome
//    d. [if refusalRetryEnabled] Post-inference refusal check:
//       → text = extractResponseText(response)
//       → result = delegate.classifyRefusal(text)   // ONNX inference
//       → emit 'refusal-classification' event
//       → if result.isRefusal:
//            emit 'refusal-detected', report REFUSAL outcome
//            skip model, retry with next model (same as throttle path)
//    e. delegate.reportOutcome(outcome)
//       → updateModelStats(), recordLatency()
//       → healthCheckManager.recordSuccess/Failure()
//       → publish 'success' or 'error' event, publish 'stats' event
//    f. On RateLimit, CircuitOpen, or Refusal: skip model, retry with next
//    g. On FailFast: raise to caller immediately
// 4. Response returned to caller (or error raised)
```